Even-Index Representations and Recursive Decomposition of Rectilinear Polygons

Wai-Shing Luk

***Abstract*—Rectilinear polygons are the native geometry of VLSI physical design, where coordinates are integers and the individual polygons are small. We study the even-index representation, which stores every other boundary vertex and recovers the omitted vertices by an x-first rule. We define the local structural condition satisfied by the encoding of a simple polygon, prove the compressed area formula, and present a monotone-hull pass (which also repairs raw creation orderings) with a betweenness-safe collinear cleanup. We give a three-step rectangle decomposition: convex decomposition, handling of stored 270° vertices, and handling of the 270° vertices introduced by the expansion. We prove that the convex stage returns orthogonally convex pieces and that the rectangle stage returns a partition into rectangles. The decomposition runs in $O(n^2)$ time, where n is the number of stored points. The algorithms are implemented in C++20. The C++ suite passes 142 cases with 962 assertions. On a twelve-point fixture the convex stage returns three pieces of areas 1, 9, and 10, and the rectangle stage returns seven rectangles of areas 1, 1, 2, 2, 4, 4, and 6, all summing to 20. The evaluation asserts the output class, not only the area, because an area check alone can accept a non-rectangle piece.**

I. Introduction

A modern integrated circuit is a physical object with billions of components, and physical design turns an abstract netlist into the polygons that a foundry can fabricate [1], [2], [3]. The geometry of the layout is overwhelmingly *rectilinear*: every edge is horizontal or vertical, and every coordinate is an integer. Rounding is not merely imprecise but illegal, because two edges that should be collinear may miss and a design-rule check fails. At the same time, the individual objects are small: a standard-cell polygon has at most about one hundred vertices, a net a handful of pins, and a block a score of metal layers. Quadratic algorithms are therefore acceptable when they are exact, simple, and easy to verify.

This paper is about the *representation* of a rectilinear polygon and the algorithms that consume it. The usual representation is the full boundary, a cyclic list of all n vertices. Because consecutive edges alternate between horizontal and vertical, every other vertex determines the omitted one under a suitable convention. The resulting *even-index* representation stores only $m = n/2$ points. It is compact, but it is also delicate: the expansion convention must be fixed, and a stored list can be degenerate or non-admissible if consecutive points share an axis or coincide.

The input throughout this paper is a **simple rectilinear polygon** P, and S is its even-index encoding. For such an encoding, consecutive stored points differ in both coordinates and no stored point is repeated; we call this the *local structural condition*. It is a property of the encoding, not a validity test, and we do not need a global simplicity check: simplicity is an assumption on the input, and the cut and cleanup operations preserve it. The proofs below therefore work directly with simple input polygons and their encodings.

We make the following contributions.

1. We define the x-first even-index encoding and the local structural condition satisfied by the encoding of a simple polygon, and we prove the compressed area formula (Section III).
2. We present a creation pass and a monotone-hull pass for stored sequences. The hull pass computes x- and y-monotone hulls and, by composition, the rectilinear convex hull; it also repairs raw creation orderings. It removes stored vertices that violate monotonicity and applies a betweenness-safe collinear cleanup; we prove boundary and area preservation under the betweenness condition and analyze the resulting $O(m^2)$ worst-case cleanup cost (Section IV).
3. We give a recursive cut framework for convex decomposition, with precise predicates, helper operations, the orientation-dependent sign test, the nearest-edge search, and the degenerate two-node split (Section V).
4. We give a rectangle decomposition in three steps: convex decomposition, handling of stored 270° vertices, and handling of the 270° vertices introduced by the expansion; we prove that it returns a partition into rectangles (Section VIII).
5. We implement the algorithms in C++20, report the exact test counts and commands, and show that output-class assertions catch non-rectangle outputs that an area-only test would accept (Section X). The C++ suite passes 142 cases with 962 assertions.

The terms used in these statements are defined before they are used: the representation and local condition in Section III; the repair operations in Section IV; and the cut operations and their helper functions in Section V.

The remainder of the paper is organized as follows. Section II reviews related work. Section III fixes notation and states the local condition. Section IV presents creation, monotone hulls, and repair. Section V develops the recursive cut framework. Section VI develops convex decomposition, and Section VIII develops the three-step

rectangle stage. Section X reports the implementation and tests, Section XI discusses scope and threats to validity, and Section XII concludes.

II. Related Work

Rectilinear polygons are a classical topic in computational geometry and a workhorse of VLSI layout. The textbook treatment covers monotone polygons, convex decomposition, and rectilinear convex hulls [4], [5], [6]. The VLSI literature uses the same primitives for layout fracturing and for decomposing L-shaped obstacles into rectangles; double-patterning and resolution-enhancement flows make rectangle decomposition a manufacturing requirement rather than a convenience [7], [8].

The optimization literature on rectangle and convex decomposition is extensive. Ohtsuki [9] gave an $O(n^{5/2})$ algorithm for minimum dissection of rectilinear regions. Lipski [10] developed a Manhattan-path approach that yields an $O(n^{3/2} \log n)$ minimum dissection, and Nahar and Sahni [11] gave a fast horizontal-cut decomposition. Soltan and Gorpinevich [12] gave an $O(n^{3/2} \log n)$ algorithm for polygons with arbitrary holes. For convex decomposition, Keil [13] gave a polynomial dynamic-programming algorithm for simple polygons, while Lingas [14] showed that minimum convex partition becomes NP-hard in the presence of holes. Keil's survey [6] and the textbook of de Berg et al. [5] provide the standard background.

Compact layout representations are also classical. Corner stitching [15] stores rectangles and their neighbors rather than a full polygon boundary. The even-index representation studied here is a different compact encoding: it halves the storage of a boundary, makes translation an $O(1)$ update of an origin in the implementation, and turns the representation into a local structural condition. It is not a new geometric object. Its interest is algorithmic: it leads to small base cases and a compact representation of the boundary.

Exact integer geometry avoids the rounding errors of floating-point predicates. Symbolic perturbation [16] is a complementary technique for degeneracies; exact integer arithmetic avoids rounding but does not by itself remove duplicate vertices, zero-length edges, or collinear points. Our algorithms handle those cases through the local condition and the betweenness condition in the cleanup.

The present work differs from the optimization literature in two ways. First, it focuses on a compressed even-index representation and on the implementation of the cut primitives on that representation. Second, it develops a rectangle decomposition on top of the convex decomposition, so that the output is a set of rectangles rather than only convex pieces. The pipeline is deliberately simple; it makes no optimality claim and is intended as a reliable preprocessing step.

III. Preliminaries and Representation

A. Rectilinear Polygons and Orthogonal Convexity

A *rectilinear* (or *orthogonal*) polygon is a simple closed polygon whose edges are axis-aligned. Its vertices alternate between horizontal and vertical edges, so it has an even number of vertices. For a polygon with holes, make a cut (a *bridge*) from the outer boundary to each hole. A zero-width bridge is traversed in both directions, so the area is unchanged and the boundary is *weakly simple* (a narrow bridge gives a simply connected polygon). The even-index encoding applies to this boundary; repeated bridge vertices are handled by the degenerate split of Section V. This reduction is standard and is not specific to the even-index representation. We state the proofs for a simply connected input; the bridge reduction supplies that case. We also assume the boundary is reduced, i.e. it has no 180° vertices; a collinear vertex is removed before the representation is built.

A rectilinear polygon is *orthogonally convex* (or *rectilinearly convex*) if every horizontal and every vertical line intersects it in a connected set, or in the empty set. Equivalently, it is both x-monotone and y-monotone. Orthogonal convexity is not Euclidean convexity: an L-shape is orthogonally convex, although its interior angle at the inner corner is 270°. We use "convex" only in the orthogonal sense in this paper and write "orthogonally convex" when the distinction matters.

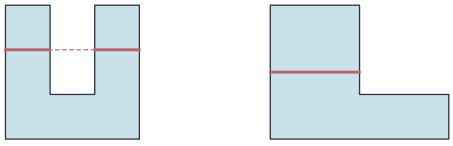


Figure 1: Left: an L-shape is both x- and y-monotone and is therefore orthogonally convex. Right: a U-shape is x-monotone but a horizontal line meets it in two intervals, so it is not orthogonally convex.

B. Even-Index Encoding

Let $v_0, \ldots, v_{n-1}$ be the vertices of a simple rectilinear polygon in boundary order, with n even. Rotate the list so that the edge from v_0 to v_1 is horizontal. The *even-index encoding* stores

$$S = (s_0, \ldots, s_{m-1}) = (v_0, v_2, \ldots, v_{n-2}), \qquad m = n/2. \tag{1}$$

The omitted odd-index vertices are implicit. With the x-first convention used throughout this paper, the implicit vertex between $s_i = (x_0, y_0)$ and $s_{i+1} = (x_1, y_1)$ is

$$I_i = (x_1, y_0), \tag{2}$$

inserted before s_{i+1}, including the closing pair $s_{m-1} \to s_0$. The resulting cyclic sequence is $E(S)$. The diagonal chord $s_i s_{i+1}$ is not a boundary edge; the boundary follows the horizontal edge to I_i and then the vertical edge to s_{i+1}.

The y-first form (x_0, y_1) reconstructs a different polygon and is not used.

For an encoding of a simple polygon, consecutive stored points differ in both coordinates, because the omitted vertex between them is a genuine corner. A two-point representation is a rectangle; a three-point representation is an L-shaped monotone piece; more points describe a larger orthogonally convex polygon. The stored order follows the boundary, and the full boundary is reconstructed on demand. The implementation stores the first stored point as an origin and the remaining points as displacement vectors, so translating the polygon updates the origin in $O(1)$. Figure Fig. 2 shows stored and implicit vertices on an eight-vertex U-shape.

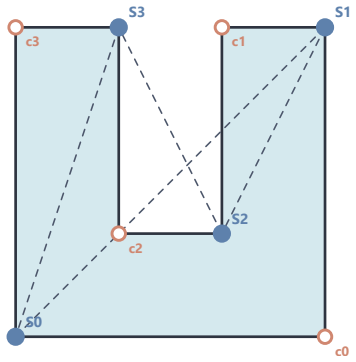


Figure 2: Stored (filled) and implicit (hollow) vertices of an eight-vertex U-shape. Dashed segments connect consecutive stored points; they are not polygon edges. The compressed representation has four stored points.

C. Local Condition

Definition 1 (local condition). A stored sequence $S = (s_0, \ldots, s_{m-1})$ is *locally admissible* if

1. $m \geq 2$,
2. the points s_i are pairwise distinct, and
3. for every i (indices modulo m),

$$x_i \neq x_{i+1} \quad \text{and} \quad y_i \neq y_{i+1}. \tag{3}$$

The condition is structural: it constrains the encoding, not the geometry. Algorithm 1 checks it directly. The quantity $m < 2$ is rejected, because a one-point list would expand to a single repeated vertex and an empty list does not describe a polygon at all.

The duplicate scan is $O(m^2)$ in the worst case; sorting the points reduces it to $O(m \log m)$. The consecutive-pair scan is $O(m)$.

Lemma 1 (the encoding satisfies the local condition). If P is a simple rectilinear polygon and S is its even-index encoding, then S is locally admissible.

Algorithm 1 IsAdmissible(S)

Require: stored points $S = (s_0, \ldots, s_{m-1})$
if $m < 2$ **then**
  return False
end if
for $i \leftarrow 0$ **to** $m - 1$ **do**
  for $j \leftarrow i + 1$ **to** $m - 1$ **do**
    if $s_i = s_j$ **then**
      return False
    end if
  end for
end for
for $i \leftarrow 0$ **to** $m - 1$ **do**
  $j \leftarrow (i + 1) \bmod m$
  if $x_i = x_j$ **or** $y_i = y_j$ **then**
    return False
  end if
end for
return True

Proof. The boundary of a simple rectilinear polygon has at least four vertices, so $m = n/2 \geq 2$. The vertices are distinct, hence the stored points are distinct. Between two consecutive stored points s_i and s_{i+1}, the omitted vertex I_i is a genuine corner: the edge from s_i to I_i is horizontal and the edge from I_i to s_{i+1} is vertical, so $x_i \neq x_{i+1}$ and $y_i \neq y_{i+1}$. The closing pair is handled by the same argument. □

Table I lists the local-condition cases used by the test suite.

D. Area

The area of the polygon can be computed from the reconstructed boundary with the shoelace formula

$$A(S) = \frac{1}{2} \sum_{i=0}^{|B|-1} (x_i y_{i+1} - x_{i+1} y_i), \tag{4}$$

where $B = E(S)$ and indices are modulo $|B|$. The implementation uses the equivalent compressed form

$$A(S) = v_1^x v_1^y + \sum_{i=2}^{m-1} v_i^x \left(v_i^y - v_{i-1}^y\right), \qquad v_i = s_i - s_0. \tag{5}$$

For a two-point rectangle the formula reduces to $(x_1 - x_0)(y_1 - y_0)$. The sign is positive for a counter-clockwise polygon and negative for a clockwise one; the orientation flag carries this sign through all algorithms.

Lemma 2 (area). For a simple rectilinear polygon, the compressed formula (5) equals the signed area of the reconstructed boundary.

Proof sketch. Expand the shoelace sum over the reconstructed sequence $s_0, I_0, s_1, \ldots$. Each implicit vertex contributes terms that cancel with the adjacent stored vertices, leaving the telescoping expression above. A direct expansion for the three-point L-shape and the two-point rectangle verifies the base cases; the general case follows by induction on m. □

Table I: Local-condition test cases.

Stored points	Locally admissible	Reason
$(2,1),(2,0)$	no	same x
$(2,1),(3,-3)$	yes	rectangle corners
$(2,1),(2,0),(3,-3)$	no	zero change in x
$(0,0),(0,0)$	no	duplicate
$(0,0),(0,1),(0,2)$	no	vertical run
$(0,0),(1,0),(2,0)$	no	horizontal run
$(0,0),(1,1)$	yes	rectangle
$(0,0)$	no	fewer than two points

Algorithm 2 CreateMonotone(P, *dir*, *cmp*)

Require: point set P, direction key *dir*, comparison *cmp*
Ensure: an ordering of P
1: $p_{\min} \leftarrow \arg\min_{p\in P} dir(p)$; $p_{\max} \leftarrow \arg\max_{p\in P} dir(p)$
2: split P into P_1 and P_2 according to the side of $p_{\min}$
3: sort P_1 by *dir*; sort P_2 by *dir* in reverse
4: **return** P_1 concatenated with P_2

E. Base Cases

The compressed representation turns the two smallest decompositions into base cases: two stored points are a rectangle, and three stored points are an L-shaped monotone piece. These are not degenerate cases; they expand to four and six boundary vertices, respectively. Since the input polygon is simple, both base cases are simple and the local condition holds by Lemma 1.

IV. Creation, Monotone Hulls, and Repair

A. Creation

The creation pass takes a point set and returns an ordering of the points that is intended to be a monotone chain. For an x-monotone ordering, sort the points by (x, y) and split them into a chain below the leftmost point and a chain above it; concatenate the lower chain with the reversed upper chain. The y-monotone version uses (y, x) as the sort key. Algorithm 2 states the shared template.

The split is deterministic. Write $dir(p) = (dir_1(p), dir_2(p))$. If the caller's comparison selects the lower chain, then

$$P_1 = \{p \in P : dir_2(p) \le dir_2(p_{\min})\}; \tag{6}$$

otherwise the inequality is reversed. In both cases $P_2 = P \setminus P_1$. The algorithm sorts P_1 ascending by *dir* and P_2 descending, and returns the concatenation P_1P_2. This is exactly the ordering used by the implementation; it is a chain order, not yet a polygon.

The creation pass returns an *ordering*, not necessarily a locally admissible list. Figure Fig. 3 shows the two orderings produced by the x-monotone and y-monotone creation routines on the twelve-point fixture used by the test suite. Both orderings contain consecutive collinear triples and therefore fail the local condition. The monotone-hull pass and the collinear cleanup turn such an ordering into a locally admissible representation.

Algorithm 3 MonotoneHull(S, σ, *dir*)

Require: stored points S, orientation σ, sweep direction *dir*
Ensure: a monotone repair, or a failure flag if the cleanup cannot produce an admissible simple list
1: $v_{\min} \leftarrow \arg\min_{v\in S} dir(v)$; $v_{\max} \leftarrow \arg\max_{v\in S} dir(v)$
2: build a circular linked list over S
3: **for** each chain $v_{\min} \to v_{\max}$ and $v_{\max} \to v_{\min}$ **do**
4: $c \leftarrow$ first vertex of the chain
5: **while** $c \neq$ stop vertex **do**
6: $p \leftarrow prev(c)$; $n \leftarrow next(c)$
7: **if** $dir(c) > dir(n)$ **or** $dir(p) > dir(c)$ **then**
8: $\Delta \leftarrow (y_c - y_p)(x_n - x_c)$
9: **if** Δ has the wrong sign for σ **then**
10: detach c; $c \leftarrow p$
11: **else**
12: $c \leftarrow n$
13: **end if**
14: **else**
15: $c \leftarrow n$
16: **end if**
17: **end while**
18: **end for**
19: **return** CleanupCollinear(S)

B. Monotone Hull

The input to the hull pass is a stored sequence S and an orientation flag $\sigma \in \{\text{CLOCKWISE}, \text{ANTICLOCKWISE}\}$. The pass finds the minimum and maximum vertices in the sweep direction, builds a circular doubly linked list, and walks the two chains from minimum to maximum and back. At a vertex with previous, current, and next points p, c, n, it computes

$$\Delta = (y_c - y_p)(x_n - x_c). \tag{7}$$

A vertex is removed when the sweep coordinate violates monotonicity and Δ has the wrong sign for the orientation. Removal is $O(1)$ in the linked list. Algorithm 3 states the procedure.

Proposition 1 (termination and cost). Algorithm 3 terminates. Each sweep is $O(m)$; the cleanup is $O(m^2)$ in the worst case, so the composed repair is $O(m^2)$.

Proof. Every detach strictly decreases the number of stored vertices, so the loop terminates. Each vertex is visited $O(1)$ times per chain and each detach is $O(1)$, hence $O(m)$ per sweep. The cleanup may remove $O(m)$ points and restart an $O(m)$ scan after each removal, hence $O(m^2)$ worst case. □

The loop stops only when no stored vertex violates the monotonicity condition, so each surviving chain is monotone at termination. The cleanup preserves the boundary and the area by Lemma 3. The proposition therefore

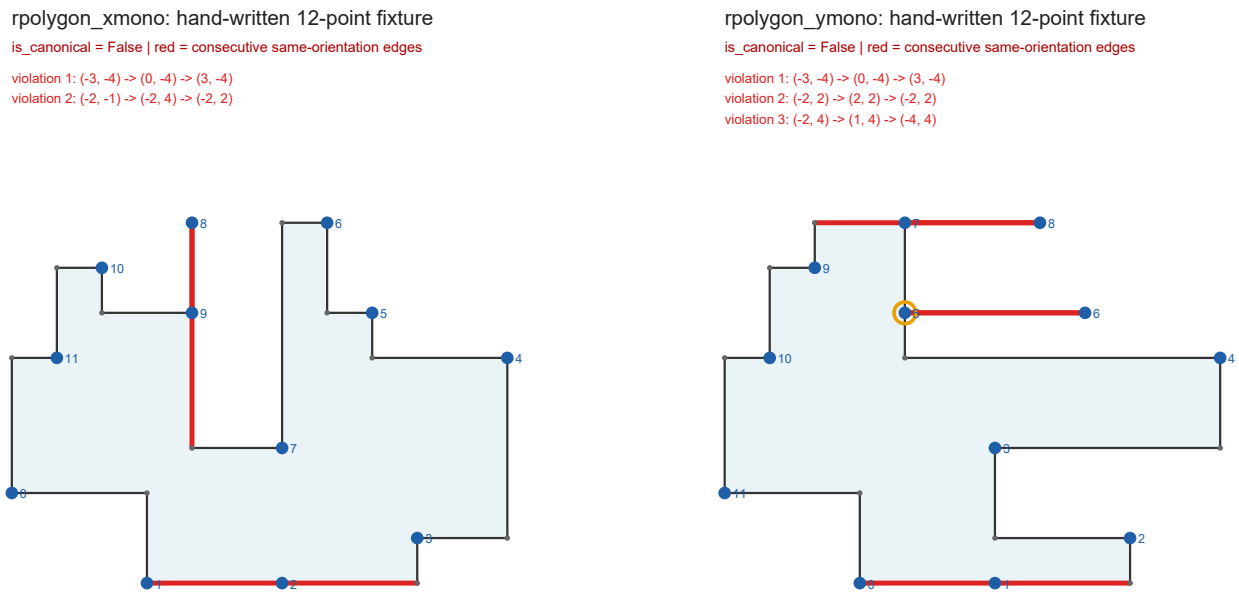


Figure 3: Non-admissible outputs of the creation routines on the twelve-point fixture: (left) the x-monotone ordering; (right) the y-monotone ordering. Red segments are consecutive collinear edges. Both lists fail the local condition.

Algorithm 4 CLEANUPCOLLINEAR(S)

Require: stored points S
Ensure: a cleaned list and a local-condition flag

changed ← TRUE
while *changed* **and** $|S| > 2$ **do**
  changed ← FALSE
  for $i \leftarrow 0$ **to** $|S| - 1$ **do**
    $j \leftarrow (i + 1) \bmod |S|$
    if $y_i = y_j$ **then**
      $k \leftarrow (j + 1) \bmod |S|$
      if $\min(x_i, x_k) < x_j < \max(x_i, x_k)$ **then**
        remove s_j; *changed* ← TRUE; **break**
      end if
    else if $x_i = x_j$ **then**
      $k \leftarrow (i - 1) \bmod |S|$
      if $\min(y_k, y_j) < y_i < \max(y_k, y_j)$ **then**
        remove s_i; *changed* ← TRUE; **break**
      end if
    end if
  end for
end while
return $(S,$ ISADMISSIBLE$(S))$

gives termination, monotonicity, and the $O(m^2)$ cost; no minimality claim is needed.

C. Betweenness-Safe Collinear Cleanup

A repair can still contain collinear stored points. In the x-first encoding, two consecutive points with the same y make the second point lie on a horizontal edge, while two with the same x make the first point lie on a vertical edge. Removing such a point preserves the represented boundary only when the point lies *strictly between* its neighbours. Algorithm 4 enforces that condition.

Lemma 3 (cleanup preserves the boundary). If the removed point lies strictly between its neighbours, Algorithm 4 preserves the represented boundary and the signed area. If it does not lie between them, the algorithm does not remove it; the returned flag reports that the input is not admissible.

Proof. A removed point lies in the interior of a straight boundary edge, so deleting it merges two collinear edges into a single edge; the polygon is unchanged as a point set and its area is unchanged. The betweenness test is exactly the condition that the point is interior to that edge. Without the test, a spike such as $((0,0),(2,0),(1,1))$ would lose the segment from $(1,0)$ to $(2,0)$; the spike has the same area but a different boundary. The flag is necessary because a vertical or horizontal run can collapse to two points that still share an axis; such a list is not admissible. □

The cleanup is a repair, not a validity certificate: it can only remove points whose removal is boundary-preserving, and it reports failure otherwise. The The wrapper raises `ValueError` when the flag is false, rather than returning a silently non-admissible list. The composed repair is therefore $O(m^2)$ worst case, which is appropriate for $m \leq 100$ but is not an $O(m)$ hull.

D. Composition, Repair, and Applications

The hull pass has two uses. The first is independent of creation: it computes the x- or y-monotone hull of a stored polygon or point set, and composing the two passes yields the rectilinear convex hull. The same construction gives a monotone enclosure or bounding shape and is a useful preprocessing step for decomposition. The second use is repair: when the input is a raw creation ordering, the passes

remove the monotonicity violations and the cleanup turns the result into a locally admissible encoding.

Applying the x-monotone hull and then the y-monotone hull produces a representation that is both x- and y-monotone on the tested fixtures; the composition is checked for monotonicity and the local condition before it is used as a decomposition input. Figure Fig. 4 shows the resulting rectilinear convex hull of a point set.

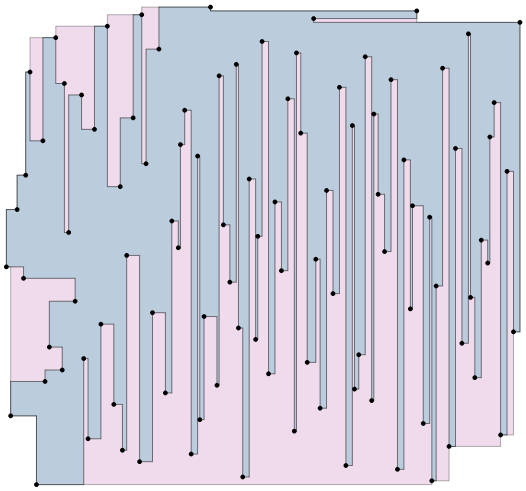


Figure 4: The rectilinear convex hull of a point set, obtained by the x- then y-monotone hull passes. The same passes serve as a repair for raw creation orderings.

V. Recursive Cut Framework

A. Data Structure

All cut policies operate on a circular doubly linked list whose nodes are the stored points. A cut inserts a new node into the list and appends the corresponding point to a point array. The recursive procedure returns lists of node indices; the wrapper converts each index list into a polygon. The linked list supports insertion and traversal in constant and linear time, respectively, and avoids the index shifting that an array-based implementation would require.

B. Concave Predicate

For three consecutive stored points p_0, p_1, p_2, the explicit and convex policies use the direction-reversal guard

$$R(p_1) = \left[(p_1^x - p_0^x)(p_2^x - p_1^x) < 0\right] \vee \left[(p_1^y - p_0^y)(p_2^y - p_1^y) < 0\right] \tag{8}$$

and the signed-area term

$$a(p_1) = (p_1^y - p_0^y)(p_2^x - p_1^x). \tag{9}$$

The vertex is treated as concave when $R(p_1)$ holds and

$$\mathrm{cmp}(a) = \begin{cases} a > 0, & \text{if the polygon is counter-clockwise,} \\ a < 0, & \text{if the polygon is clockwise.} \end{cases} \tag{10}$$

This is the compressed-list form of the reflex test. It should not be read as a test for every reflex vertex of the reconstructed boundary. The inner corner of an L-shape is a reflex vertex of the full boundary, yet the compressed predicate does not cut it, because the L-shape is already orthogonally convex. The predicate detects a stored corner at which the compressed chain fails the monotonicity condition; its role is to drive the partition into orthogonally convex pieces, not to enumerate all 270° corners.

Algorithm 5 FindConcave(v, π)

Require: root v, policy π
$c \leftarrow v$
while True **do**
  $p_0 \leftarrow prev(c)$; $p_1 \leftarrow c$; $p_2 \leftarrow next(c)$
  if Accept$_\pi(p_0, p_1, p_2)$ **then**
    return c
  end if
  $c \leftarrow next(c)$
  if $c = v$ **then**
    return nil
  end if
end while

Algorithm 6 FindMinDist(v)

Require: concave vertex v
$best \leftarrow \infty$; $target \leftarrow$ nil; $vertical \leftarrow$ True
for each stored node q **do**
  $q^- \leftarrow prev(q)$; $q^+ \leftarrow next(q)$
  if $q_y^- < v_y \le q_y$ **or** $q_y \le v_y < q_y^-$ **then**
    $d \leftarrow |q_x - v_x|$
    if $d < best$ **then**
      $best \leftarrow d$; $target \leftarrow q$; $vertical \leftarrow$ True
    end if
  end if
  if $q_x^+ < v_x \le q_x$ **or** $q_x \le v_x < q_x^+$ **then**
    $d \leftarrow |q_y - v_y|$
    if $d < best$ **then**
      $best \leftarrow d$; $target \leftarrow q$; $vertical \leftarrow$ False
    end if
  end if
end for
return $(target, vertical)$

C. Helper Operations

The skeleton uses two helper operations. We write v_x, v_y for the coordinates of a point v, and q^-, q^+ for the predecessor and successor of a stored point q. FindConcave scans stored triples in cyclic order and returns the first vertex accepted by the policy. FindMinDist scans stored points and their incident edges, projects the concave vertex onto each edge that spans its coordinate, and returns the nearest projection together with its direction.

Table II defines the three policy predicates and their contracts. The convex and explicit policies use the stored-triple guard (8) and area term (9); the implicit policy uses the stored-pair sign test (15). The explicit policy is defined for a general simple polygon; the implicit policy is documented for a convex input, which is the contract used in Section VII.

D. Skeleton

Algorithm 7 shows the shared skeleton. It is a template method: the policies differ in the base cases, in the concave-vertex predicate, and in the target of the first recursive call.

Table II: Cut policies. The convex and explicit predicates use $R(p_1)$ and $a(p_1)$ from (8)–(9); the implicit predicate uses (15).

Policy	Predicate	Base case	First target	Input contract
Convex	$R(p_1) \wedge \mathrm{cmp}(a(p_1))$	2 or 3	v_{cur}	simple polygon
Explicit	$R(p_1) \wedge \mathrm{cmp}(a(p_1))$	2	v_{cur}	simple polygon
Implicit	$\mathrm{cmp}(a(c_i))$	2	v_{min}	convex input

Algorithm 7 CUT(v)

Require: a circular list of stored points rooted at v
Ensure: a list of pieces
if v has exactly two nodes **then**
  return the two-node piece
end if
if the policy allows a three-node base case and v has three nodes **then**
  return the three-node piece
end if
$v_{\mathrm{cur}} \leftarrow$ FINDCONCAVE(v)
if $v_{\mathrm{cur}} =$ NIL **then**
  return the whole cycle as one piece
end if
$(v_{\mathrm{min}}, \mathit{vertical}) \leftarrow$ FINDMINDIST(v_{cur})
if a degenerate two-node split applies **then**
  return SPLIT($v_{\mathrm{cur}}, v_{\mathrm{min}}$)
end if
$p_{\mathrm{new}} \leftarrow$ INSERTCUT($v_{\mathrm{cur}}, v_{\mathrm{min}}, \mathit{vertical}$)
$u \leftarrow$ FIRSTTARGET($v_{\mathrm{cur}}, v_{\mathrm{min}}$)
return CUT(u) $\cup$ CUT(p_{new})

The convex policy is described in Section VI; the explicit and implicit policies reuse the skeleton with different predicates in Section VII.

The base cases are stored pieces, not expanded boundaries. A two-node piece is a rectangle; a three-node piece is an L-shape when the expansion is simple. The predicate `FindConcave` returns the first stored vertex that violates the policy's concavity test. If no such vertex exists, the current cycle is returned as a single piece. The nearest-edge search returns a node v_{min} and a flag indicating whether the cut is vertical. The cut is inserted as a new node; the two recursive calls then process the two sides. The `FirstTarget` hook is the only difference between the explicit and implicit recursions: the explicit policy recurses first on v_{cur}, the implicit policy on v_{min}.

E. Nearest Edge and Cut Insertion

Given a concave vertex v, the algorithm projects v vertically and horizontally onto the boundary and selects the nearest edge. For the explicit and convex policies the projection point is v itself; for the implicit policy it is the implicit corner $c = (x_{\mathrm{next}}, y_{\mathrm{cur}})$. The search examines each edge in the circular list and keeps the minimum distance. If the nearest edge is vertical, the new cut point is

$$p_{\mathrm{new}} = (x_{\mathrm{min}}, y_{\mathrm{cur}}); \tag{11}$$

if it is horizontal,

$$p_{\mathrm{new}} = (x_{\mathrm{cur}}, y_{\mathrm{min}}). \tag{12}$$

The new node is spliced between the appropriate neighbours. The update replaces the target edge by two edges through p_{new} and links p_{new} to the concave vertex, so the circular list splits into two cycles that share p_{new} and v; the four affected links are updated in $O(1)$. The projection condition in the search is the discrete visibility test: it selects an edge that spans the coordinate of v, so the segment from v to the projection point lies inside the polygon. The implementation uses a strict inequality to break ties and to avoid selecting an edge incident to v itself. Lemma 4 proves that the projection is the first boundary hit and that the open segment lies inside the polygon.

F. Degenerate Cuts

A cut point can coincide with an existing vertex. When the nearest target is adjacent to the current vertex in the circular list and the cut point duplicates that target, inserting a new node would create a zero-length edge. The implementation detects this situation and splits off the existing two-node piece instead of inserting a duplicate. The twelve-point fixture exercises this case: the first cut at the stored vertex $(3, 2)$ targets $(1, 2)$, which is adjacent in the list, so the two-node piece $(2, 3), (1, 2)$ is split off as a unit. The remaining two pieces are the area-10 piece (six stored points in the raw output, five after the cleanup) and the five-point piece of area 9. Without the degenerate split, the same run returns a single nineteen-area piece and a one-area piece, and the three-piece partition is lost.

The convex wrapper can also receive a zero-width two-node sliver on a mixed fixture. Such a sliver has zero area and is discarded by the wrapper; it is not a piece of the output. Because the sliver has empty interior, discarding it does not change the partition of the polygon's interior or the area sum. The the test suite records this behavior.

VI. Convex Decomposition

A. Algorithm

The convex policy cuts until no stored triple satisfies the predicate of Section V. Its base cases are the two-node rectangle and the three-node monotone piece. The recursion is the skeleton of Algorithm 7 with the convex predicate and `has_triangle_base_case = true`.

B. Correctness

Lemma 4 (cut partition). A cut from a concave vertex to the first boundary point hit by an interior axis-parallel ray splits a simple rectilinear polygon into two simple rectilinear polygons whose interiors are disjoint and whose areas sum to the area of the original polygon.

Proof. Let v be the concave vertex and let p be the first boundary point hit by the ray from v into the interior. Such

a point exists because the polygon is bounded. The open segment (v, p) cannot meet the boundary: if it did, the first such intersection would be closer to v than p; if it left the interior, it would cross the boundary at a closer point. Hence (v, p) lies in the interior. The nearest-edge scan of Algorithm 6 considers exactly the edges that cross the ray. For a vertical target edge the distance along the ray is $|q_x - v_x|$, and for a horizontal target edge it is $|q_y - v_y|$; the minimum over the scanned edges is therefore the first hit p. The strict inequalities in the scan exclude the two edges incident to v. Adding the segment vp to the boundary produces two closed cycles. Each cycle is simple because vp meets the boundary only at its endpoints and does not cross the boundary in its interior. The interiors of the two cycles are disjoint and their union is the interior of the original polygon. The shoelace sum is additive because the segment vp is traversed in opposite directions in the two cycles. □

Lemma 5 (convexity criterion). Let Q be a simple orthogonal polygon with stored sequence S. If Q is not orthogonally convex, then some stored triple satisfies the convex predicate of Section V.

Proof. If Q is not orthogonally convex, an axis-parallel line L meets Q in at least two disjoint intervals. Take two consecutive intervals and let G be the gap between them. Walk along the boundary of Q from one side of G to the other; the first reflex vertex v encountered is the innermost corner of a dent that separates the two intervals. If v is stored, let p_0 and p_2 be its stored predecessor and successor. The two incident edges at v turn in opposite coordinate senses, so $R(v)$ holds; because v is reflex, the signed area term $a(v)$ has the sign selected by cmp, so the predicate accepts v. If v is implicit, write $v = I_i$ for the stored pair s_i, s_{i+1}. The four possible orientations of the two incident edges are checked directly: at the stored endpoint s_i (or s_{i+1}) the guard R and the sign of a are exactly the sign of the omitted reflex corner, so a stored triple is accepted. Thus a non-orthogonally-convex piece always exposes an accepted stored triple. □

Theorem 1 (convex decomposition). The convex recursion terminates and returns a partition of the input polygon into orthogonally convex pieces. The area of the pieces sums to the area of the input.

Proof. Each cut is a non-crossing chord of the current simple polygon (Lemma 4). A simple polygon with n vertices admits at most $n - 3$ non-crossing chords, so the recursion performs $O(n)$ cuts and terminates; the base cases stop on two or three stored points. Lemma 4 gives the partition and the area identity. At termination no stored triple satisfies the convex predicate, so Lemma 5 implies that the piece is orthogonally convex. A zero-area sliver has empty interior and is discarded without changing the partition of the interior. □

C. Example

The twelve-point fixture has stored points

$$(-1, 1), (0, 0), (1, -1), (3, 0), (4, 1), (6, 2), (5, 5), (4, 4), (3, 2), (2, 3), (1, 2) \tag{13}$$

and area 20. The convex policy returns three pieces, listed in Table III. The two-node piece is a rectangle; the other two are orthogonally convex but not rectangles. These pieces are the input to the rectangle stage of Section VIII.

The C++ convex policy returns the six-point list shown for the area-10 piece. The cleanup pass removes the collinear stored point $(3, 0)$ from the raw six-point list, yielding the five-point list; the area and the output class are unchanged. This is a concrete example of why the cleanup is a repair pass rather than a validity check.

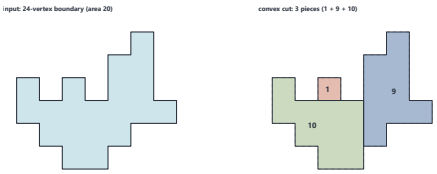


Figure 5: Convex decomposition of the twelve-point fixture: three orthogonally convex pieces of areas 1, 9, and 10.

VII. Explicit and Implicit Policies

A. Explicit Policy

The explicit policy uses the same recursive skeleton and the same concavity predicate as the convex policy, but it omits the three-node base case. It handles the stored 270° vertices: it continues until no stored triple satisfies the predicate. In the three-step rectangle algorithm, the explicit phase is applied before the implicit phase, so the stored corners are resolved first and the implicit phase only has to handle the corners introduced by the expansion.

B. Implicit Policy

The implicit policy operates directly on stored pairs. For a stored point $p_1 = S_i$ and its successor $p_2 = S_{i+1}$, the corner under test is the implicit vertex

$$c_i = (x_{i+1}, y_i). \tag{14}$$

The C++ API documents this entry point as a cut of a *convex* polygon; the policy assumes a convex input, because it tests only implicit corners. It is applied to the convex pieces produced by the convex stage.

The policy uses the sign test

$$a(c_i) = -(y_{i+1} - y_i)(x_{i+1} - x_i) \tag{15}$$

and the orientation comparison of (10). The implicit vertex is a genuine 90° corner by construction, so the sign test alone decides whether it is reflex in the orientation of the polygon.

Table III: Convex decomposition of the twelve-point fixture. The areas sum to 20.

Piece	Stored points	Area
Rectangle	$(2,3),(1,2)$	1
Orthogonally convex	$(3,2),(0,3),(-1,1),(0,0),(1,-1),(3,0)$	10
Orthogonally convex	$(3,0),(4,1),(6,2),(5,5),(4,4)$	9

For example, the L-shape stored as $((0,0),(2,1),(1,2))$ has one implicit corner, $(1,1)$. The relevant stored pair is $((2,1),(1,2))$. The test value is

$$a = -(2-1)(1-2) = +1, \tag{16}$$

which is positive, so the counter-clockwise orientation selects the implicit corner. A horizontal cut yields the rectangles $((0,0),(2,1))$ and $((0,1),(1,2))$ of areas 2 and 1.

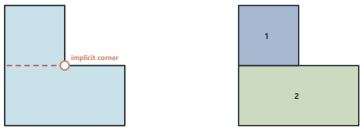


Figure 6: Implicit cut of an L-shape. The implicit corner $(1,1)$ is reflex; a horizontal cut yields two rectangles of areas 2 and 1.

C. Explicit and Implicit Corners

The explicit policy handles stored 270° vertices; the implicit policy handles omitted 270° vertices. The two cases are disjoint: a stored point is not an omitted point, and an omitted point is not stored. In the three-step rectangle algorithm of Section VIII, the explicit phase is applied first, so the stored corners are resolved before the implicit phase; the implicit phase then handles the corners introduced by the expansion. The L-shape example above shows the implicit phase; a stored reflex corner is handled by the explicit phase in the same way. Together the two phases cover both types of 270° vertex.

VIII. Rectangle Decomposition

A. Three Steps

A rectangle decomposition is specified by three steps.

1. **Convex decomposition.** Compute a partition of the input into orthogonally convex pieces. This step makes the next two steps local: each piece is x- and y-monotone, and every remaining 270° vertex lies on one of the two monotone chains. A cut at such a vertex therefore splits the piece into two x-monotone pieces.
2. **Handle explicit** 270° **vertices.** A vertex is *explicit* if it is a stored point and its interior angle is 270°. For each explicit 270° vertex, cut from the vertex to the nearest visible edge (or vertex), split the piece, and recurse. Repeat until no explicit 270° vertex remains.
3. **Handle implicit** 270° **vertices.** A vertex is *implicit* if it is an omitted vertex $I_i = (x_{i+1}, y_i)$ and its interior angle is 270°. Cut it in the same way and recurse. Repeat until no implicit 270° vertex remains.

A simple orthogonal polygon with no 270° vertex is a rectangle. Steps 2 and 3 are repeated until neither type remains; a cut can create a new 270° vertex of the other type, and the corresponding step handles it. The three steps therefore specify a rectangle decomposition.

B. Correctness

Theorem 2 (rectangle decomposition). Let P be a simple rectilinear polygon. Apply the convex decomposition of Section VI; then apply the explicit phase to every stored 270° vertex and the implicit phase to every 270° vertex introduced by the expansion, repeating the two phases until neither type remains. The output is a partition of P into rectangles, and the areas of the rectangles sum to $A(P)$.

Proof. Theorem 1 partitions P into simple orthogonal pieces. In any such piece, every 270° vertex is a boundary vertex; it is either a stored point or an omitted point I_i. Step 2 cuts the stored 270° vertices and step 3 cuts the omitted ones; repeating the two phases handles vertices created by a cut. When neither type remains, every interior angle is 90°. A simple orthogonal polygon has $n = 2r + 4$ vertices, where r is the number of 270° vertices; with $r = 0$ we have $n = 4$, and a simple orthogonal quadrilateral is a rectangle. Every cut is a chord inside the current piece (Lemma 4), so the pieces partition P and their areas add. □

The seven-rectangle result in Table IV is the output of this pipeline on the twelve-point fixture.

C. Example

Applying the pipeline to the twelve-point fixture yields seven rectangles, listed in Table IV. Every rectangle has two stored points with distinct x- and y-coordinates, and the areas sum to 20.

D. Remarks

The algorithm is not a minimum-cardinality rectangle partition. A carefully chosen cut sequence can use fewer rectangles. The goal here is a simple, deterministic, and provably correct pipeline that is easy to verify against the output class. Minimum rectangle partition is a separate optimization problem [9], [10], [12].

The implementation provides the convex, explicit, and implicit phases and the three-step rectangle pipeline. The

Table IV: Rectangle decomposition of the twelve-point fixture. The areas sum to 20.

Rectangle	Opposite corners	Area
R_1	$(1,-1),(3,2)$	6
R_2	$(0,0),(1,2)$	2
R_3	$(1,2),(2,3)$	1
R_4	$(-1,1),(0,3)$	2
R_5	$(3,0),(4,4)$	4
R_6	$(4,1),(5,5)$	4
R_7	$(5,1),(6,2)$	1

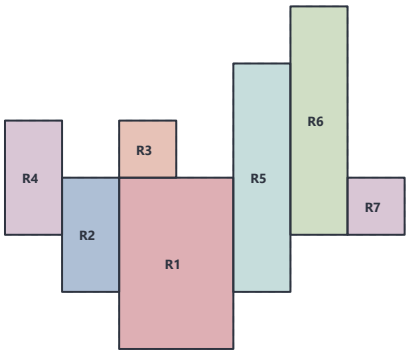


Figure 7: Rectangle decomposition of the twelve-point fixture: seven rectangles of areas 1, 1, 2, 2, 4, 4, and 6.

seven-rectangle result in Table IV and Figure Fig. 7 is produced by this pipeline.

IX. Complexity

Let n be the number of stored points of the input polygon.

Local condition. The local-condition scan is $O(n^2)$ in the simple implementation and $O(n \log n)$ after sorting. It is adequate for the small polygons of the domain.

Creation and repair. The creation sort is $O(n \log n)$. Each monotone sweep is $O(n)$; the betweenness-safe cleanup is $O(n^2)$ worst case, so the composed repair is $O(n^2)$.

Cut stage. The nearest-edge search scans $O(n)$ edges. Each cut increases the number of pieces by one and the recursion creates at most $O(n)$ pieces, so the cut stage takes $O(n^2)$ time. The linked list supports insertion in $O(1)$ and the point array is appended in amortized $O(1)$. The recursion uses $O(n)$ stack space in the worst case and $O(n)$ storage for the pieces.

Rectangle stage. Each explicit or implicit cut scans $O(n)$ stored points and edges, and the recursion creates at most $O(n)$ pieces, so the rectangle stage takes $O(n^2)$ time in the worst case. The output has $O(n)$ rectangles because every cut increases the number of pieces by one.

Overall. The pipeline runs in $O(n^2)$ time and $O(n)$ extra space. On the twelve-point fixture the convex stage returns three pieces after two cuts, and the rectangle stage returns seven rectangles.

X. Implementation and Verification

A. Implementations

The algorithms are implemented in C++20 in `physdes-cpp` (the `Recti` library). The implementation provides the local-condition check, the creation and repair passes, the convex, explicit, and implicit phases, and the three-step rectangle pipeline. It uses the twelve-point fixture and the area/orientation conventions described above.

The tests build and run with `xmake build test_recti` and `xmake run test_recti`. The suite is the `test_recti` target at commit `b5ef6ae` of `physdes-cpp`, plus the test fixtures described in this paper.

B. Test Design

The tests cover five layers.

1. **Local-condition tests.** The cases of Table I, including the $m < 2$ cases.
2. **Cleanup tests.** The spike `((0,0),(2,0),(1,1))` is not removed by the cleanup and is reported as non-admissible; vertical and horizontal runs are rejected.
3. **Creation and repair tests.** Create an ordering, apply the matching monotone repair, and assert the local condition and monotonicity on the result. The twelve-point fixture whose x-repair is not locally admissible before cleanup is included.
4. **Cut tests.** Assert orthogonal convexity on the *expanded* pieces and check signed-area preservation. The twelve-point fixture is required to return three pieces of areas 1, 9, and 10. The mixed fixture that produces a zero-width sliver is required to return two valid pieces; the sliver is discarded. The wrapper applies the cleanup before these assertions; the raw convex policy returns the six-point area-10 list described above, so the fixture test checks both its area and convexity and the cleaned five-point representative. A bridged-hole fixture $(4,1),(10,7),(7,5),(9,2),(6,5),(7,7)$ is cut into three pieces of areas $2, 15, 10$; the bridge is traversed twice and the area is preserved. Two AGPLIB instances, `rand-10-1` and `rand-12-6` [17], are cut into two and three orthogonally convex pieces with area preservation. The C++ suite includes the same two AGPLIB fixtures and the bridged-hole fixture.
5. **Rectangle tests.** Assert that every output piece is a two-point rectangle with distinct coordinates and

positive area; the twelve-point fixture must return seven rectangles of areas 1, 1, 2, 2, 4, 4, and 6, and the area sum must be 20.

The convexity oracle is deliberately not `rpolygon_is_convex(piece)` on the stored points. That call checks the stored points, not the implicit vertices, and it returns true for any stored list of length at most three. The tests expand the piece first and check orthogonal convexity on the full boundary. This is the oracle that catches a zero-length edge in the six-point cut piece.

C. Results

Table V reports the suite sizes and the fixture results. The C++ suite passes 142 cases with 962 assertions under `xmake`.

The twelve-point fixture is decomposed into seven rectangles of areas $1, 1, 2, 2, 4, 4, 6$, all summing to 20. The tests assert the output class (local condition, orthogonal convexity, and rectangle shape), not only the area; this is what distinguishes a correct rectangle decomposition from a collection of convex pieces with the right total area.

D. Threats to Validity

The evaluation uses generated point sets and hand-built fixtures, not foundry data. The claims are algorithmic: exactness, termination, area preservation, and the output class. They are not signoff claims. The implementation is not cross-validated against an independent oracle on random inputs; the shared fixture and the same area/orientation conventions are the current check. It uses `int` coordinates; the intended range is small, and overflow is not handled.

XI. Discussion and Limitations

Representation. The local condition is a property of the encoding, not a validity test; simplicity is an assumption on the input. The encoding depends on the starting vertex, and the algorithms use the given rotation. The implementation retains the name `is_canonical` for compatibility.

Polygons with holes. A polygon with holes is reduced to a single boundary by a bridge cut from the outer boundary to each hole. The bridge is traversed in both directions, so the area is unchanged; the boundary is weakly simple for a zero-width bridge and simply connected for a narrow bridge. The even-index encoding and the cut skeleton then apply to that boundary, and repeated bridge vertices are handled by the degenerate split. This is standard and is not specific to the even-index representation. The proofs are stated for a simply connected input; the bridge reduction supplies that case.

No optimality claim. The rectangle pipeline is a cut-based decomposition, not a minimum-cardinality partition. The seven rectangles on the fixture are not claimed to be minimal. Minimum rectangle partition is a different problem with a rich literature [9], [10], [12]; the goal here is a simple, verified preprocessing pipeline.

Proof scope. The proofs assume a simply connected input and a reduced boundary. They cover termination, partition, area preservation, the convexity criterion, orthogonal convexity of the convex stage, and rectangle output. A polygon with holes is reduced to this case by the bridge cut described above; minimum-cardinality optimization is not addressed.

Quadratic scaling. The decomposition is $O(n^2)$. For $n \leq 100$ this is appropriate; for $n > 10^3$ a sub-quadratic method would be preferable. The literature on minimum/near-minimum rectangle partition is relevant, but the present pipeline is not optimized for that regime.

Degeneracies. Zero-width slivers, duplicate implicit points, and collinear stored points are handled by the betweenness cleanup, the degenerate two-node split, and the zero-width discard. The treatment is not exhaustive: cuts through vertices and cuts along the boundary still need separate case analysis. Exact integer arithmetic avoids rounding but not these combinatorial degeneracies; symbolic perturbation [16] is a complementary technique.

XII. Conclusion

We studied the even-index representation of rectilinear polygons and the algorithms that create, repair, and decompose them. For a simple input polygon, we proved that the encoding satisfies the local condition, proved the compressed area formula, and proved that the betweenness-safe cleanup preserves the boundary and area. The recursive cut framework has precise convex, explicit, and implicit phases; the cut-partition lemma, the convexity criterion, and the rectangle theorem show that the three-step algorithm returns a partition into rectangles. The implementation is in C++20 and is verified by output-class tests, not only by area checks. A polygon with holes is reduced to a single boundary by a bridge cut from the outer boundary to each hole; this is not specific to the even-index representation. Open problems include minimum-cardinality rectangle partitioning.

XIII. Code Availability

The C++20 implementation is in `physdes-cpp` under the `luk036` organization. The tests build and run with `xmake build test_recti` and `xmake run test_recti`. The implementation provides the local-condition check, creation/repair, the convex, explicit, and implicit phases, and the three-step rectangle pipeline.

[1] N. A. Sherwani, *Algorithms for VLSI physical design automation*, 3rd ed. Boston, MA: Kluwer Academic Publishers, 1999.

[2] A. B. Kahng, J. Lienig, I. L. Markov, and J. Hu, *VLSI physical design: From graph partitioning to timing closure*. Dordrecht: Springer, 2011.

Table V: Verification results.

Suite / check	Cases / assertions	Result
C++ `test_recti`	142 cases, 962 assertions	pass
Local condition	Table I	pass
Convex cut, twelve-point fixture	3 pieces; areas 1, 9, 10	pass
Rectangle cut, twelve-point fixture	7 rectangles; areas 1, 1, 2, 2, 4, 4, 6	pass
Area preservation	all convex-cut and rectangle tests	pass

[3] S. M. Sait and H. Youssef, *VLSI physical design automation: Theory and practice*. Singapore: World Scientific, 1999.

[4] F. P. Preparata and M. I. Shamos, *Computational geometry: An introduction*. New York: Springer, 1985. doi: 10.1007/978-1-4612-1098-6.

[5] M. de Berg, O. Cheong, M. van Kreveld, and M. Overmars, *Computational geometry: Algorithms and applications*, 3rd ed. Berlin: Springer, 2008. doi: 10.1007/978-3-540-77974-2.

[6] J. M. Keil, "Polygon decomposition," in *Handbook of computational geometry*, J.-R. Sack and J. Urrutia, Eds., Amsterdam: Elsevier, 2000, pp. 491–518. doi: 10.1016/B978-044482537-7/50012-7.

[7] A. B. Kahng, C.-H. Park, X. Xu, and H. Yao, "Layout decomposition for double patterning lithography," in *Proc. IEEE/ACM int. Conf. Computer-aided design (ICCAD)*, 2008, pp. 465–472. doi: 10.1109/ICCAD.2008.4681616.

[8] L. W. Liebmann, "Layout impact of resolution enhancement techniques: Impediment or opportunity?" in *Proc. Int. Symp. Physical design (ISPD)*, 2003, pp. 110–117. doi: 10.1145/640000.640026.

[9] T. Ohtsuki, "Minimum dissection of rectilinear regions," in *Proc. IEEE international symposium on circuits and systems (ISCAS)*, Rome, Italy, 1982, pp. 1210–1213.

[10] Jr. Lipski Witold, "An $O(n \log n)$ Manhattan path algorithm," *Information Processing Letters*, vol. 19, no. 2, pp. 99–102, 1984, doi: 10.1016/0020-0190(84)90105-4.

[11] S. Nahar and S. Sahni, "Fast algorithm for polygon decomposition," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 7, no. 4, pp. 473–483, 1988, doi: 10.1109/43.3182.

[12] V. G. Soltan and A. Gorpinevich, "Minimum dissection of a rectilinear polygon with arbitrary holes into rectangles," *Discrete and Computational Geometry*, vol. 9, no. 1, pp. 57–79, 1993, doi: 10.1007/BF02189307.

[13] J. M. Keil, "Decomposing a polygon into simpler components," *SIAM Journal on Computing*, vol. 14, no. 4, pp. 799–817, 1985, doi: 10.1137/0214056.

[14] A. Lingas, "The power of non-rectilinear holes," in *Automata, languages and programming (ICALP)*, in Lecture notes in computer science, vol. 140. Berlin: Springer, 1982, pp. 369–383. doi: 10.1007/BFb0012784.

[15] J. K. Ousterhout, "Corner stitching: A data-structuring technique for VLSI layout tools," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 3, no. 1, pp. 87–100, 1984, doi: 10.1109/TCAD.1984.1270061.

[16] H. Edelsbrunner and E. P. Mücke, "Simulation of simplicity: A technique to cope with degenerate cases in geometric algorithms," *ACM Trans. Graph.*, vol. 9, no. 1, pp. 66–104, 1990.

[17] M. C. Couto, C. C. de Souza, and P. J. de Rezende, "AGPLIB: Art gallery problem instance library." https://www.ic.unicamp.br/~cid/Problem-instances/Art-Gallery/AGPVG/.